# Lab 7(A) - Laplacian Filter

| Name | ID |
|---|---|
| Nawaf Mohammed Alghamdi | 2240007352 |

In [1]:
```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)



%matplotlib inline
```

## Apply Laplacian Filter on Image to get Sharp Image

In [2]:

```
# ================ ( Laplacian Filter ) ====================


# 0. Image file path
imgFile = "moon.jpg"

# 1. Load image and convert to grayscale
img = cv2.cvtColor(cv2.imread(imgFile, cv2.IMREAD_COLOR),
cv2.COLOR_BGR2RGB)

# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical
edges
manual_kernel = np.array([
    [0,  1,  0],
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)

# Alternative kernel: Detects diagonal edges as well
# manual_kernel = np.array([
#     [1,  1,  1],
#     [1, -8,  1],
#     [1,  1,  1]
# ], dtype=np.float32)

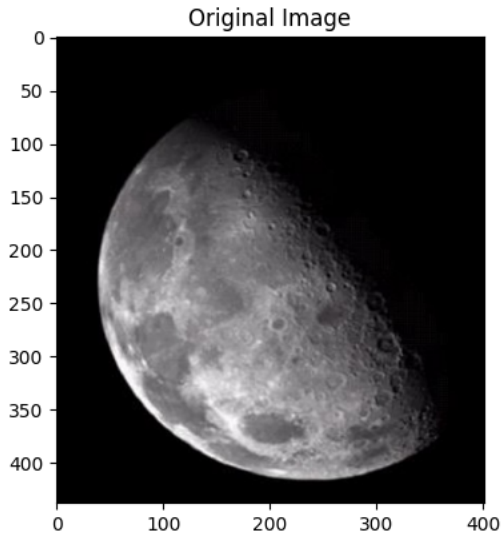
# 3. Apply the filter using filter2D
# -1 indicates the output image will have the same depth as the source
# However, for Laplacian, use cv2.CV_64F to avoid clipping negative
values
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)

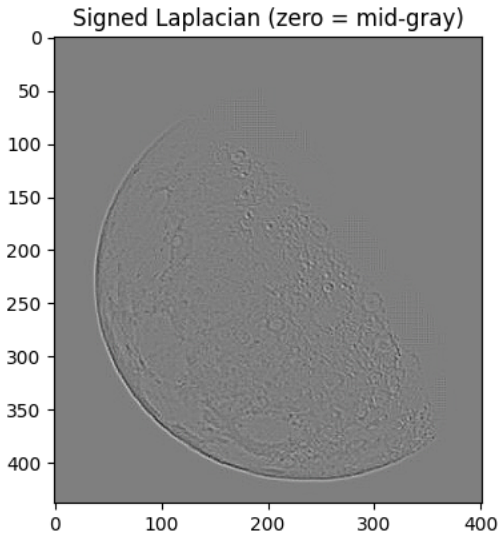
# 4. Take absolute value and convert back to 8-bit (0-255)
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)

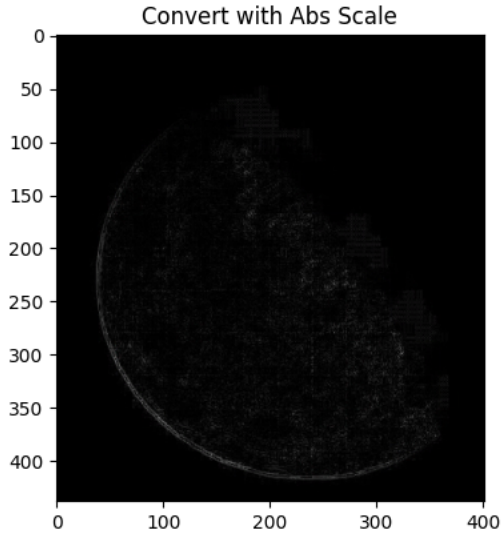
laplacian_display = np.clip(0.5 + laplacian_manual / (2 *
max(np.max(np.abs(laplacian_manual)), 1)), 0, 1)

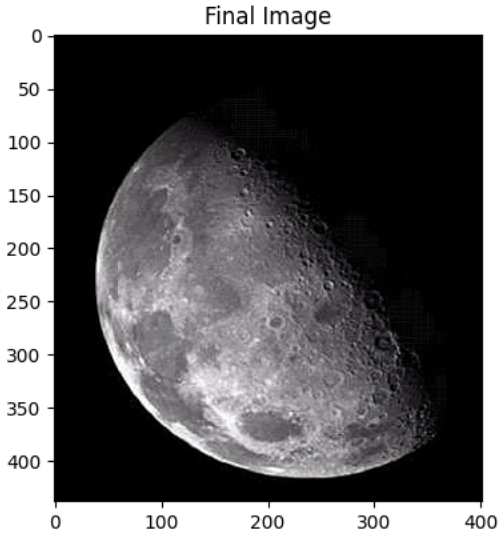
finalImage = np.clip(img.astype(np.float64) - laplacian_manual, 0,
255).astype(np.uint8)

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_display), plt.title('Signed
Laplacian (zero = mid-gray)')
plt.subplot(223), plt.imshow(laplacian_abs), plt.title('Convert with
Abs Scale')
plt.subplot(224), plt.imshow(finalImage), plt.title('Final Image')
plt.show()
```


Original Image
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Signed Laplacian (zero = mid-gray)
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Convert with Abs Scale
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Final Image
0
50
100
150
200
250
300
350
400
0
100
200
300
400

In [3]:

```
# ================== ( Laplacian Filter ) ===============

# 1. Optional: Apply Gaussian Blur to reduce noise
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)

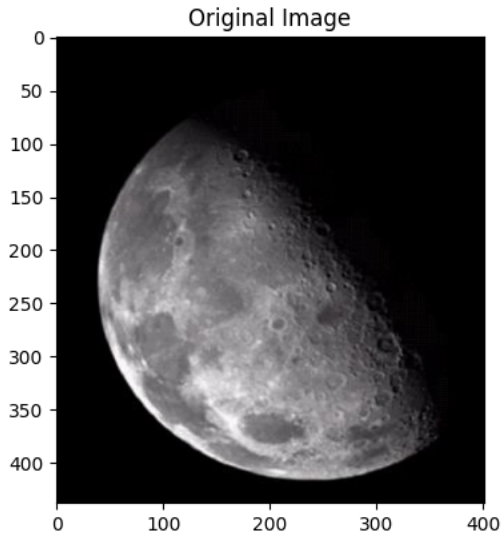
# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients
(essential for edges)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

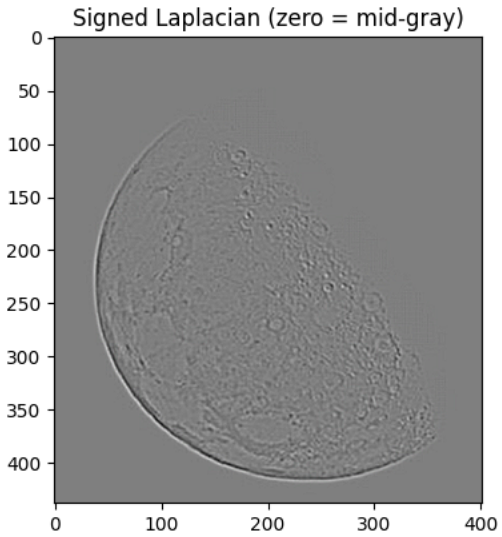
# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)

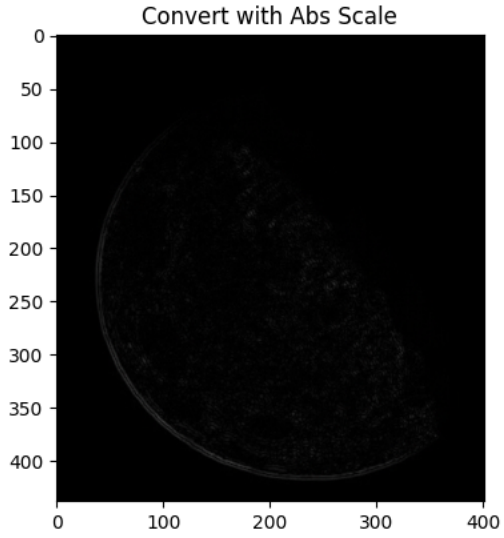
# 4. Adding Laplace and origianl Image
laplacian_display2 = np.clip(0.5 + laplacian / (2 *
max(np.max(np.abs(laplacian)), 1)), 0, 1)

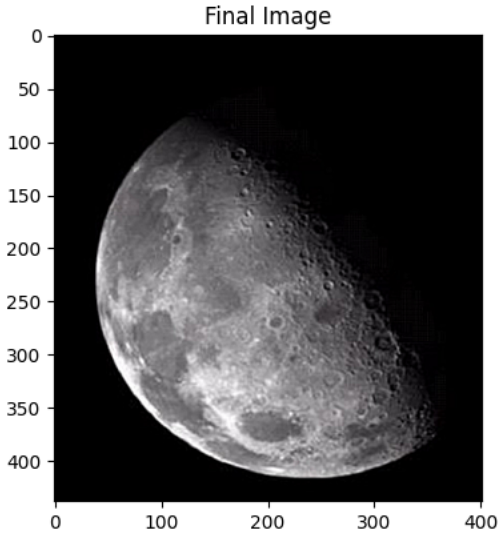
finalImage2 = np.clip(img.astype(np.float64) - laplacian, 0,
255).astype(np.uint8)

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_display2), plt.title('Signed
Laplacian (zero = mid-gray)')
plt.subplot(223), plt.imshow(laplacian_8bit), plt.title('Convert with
Abs Scale')
plt.subplot(224), plt.imshow(finalImage2), plt.title('Final Image')
plt.show()
```


Original Image
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Signed Laplacian (zero = mid-gray)
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Convert with Abs Scale
0
50
100
150
200
250
300
350
400
0
100
200
300
400


Final Image
0
50
100
150
200
250
300
350
400
0
100
200
300
400